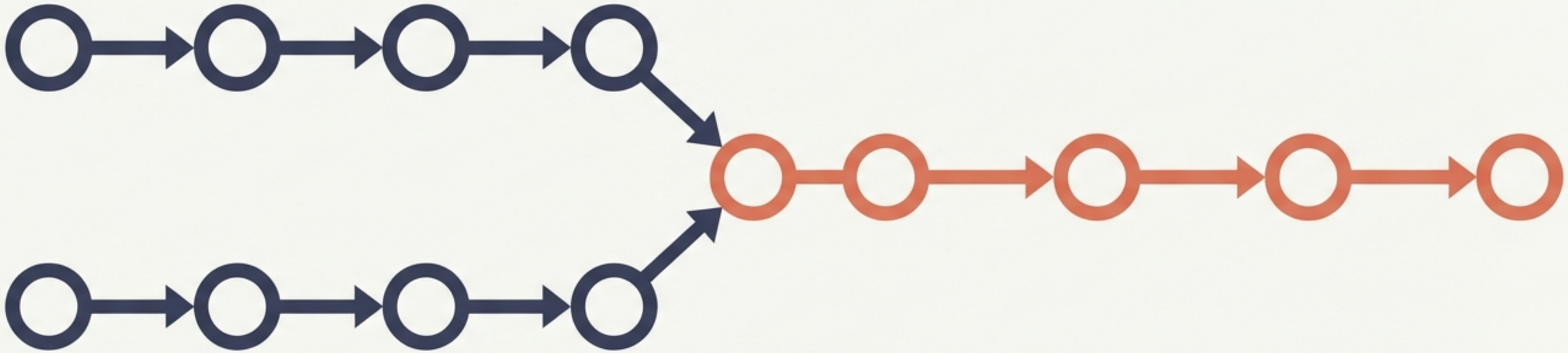
Cracking LeetCode #2: Add Two Numbers

A visual deconstruction of Linked List arithmetic.



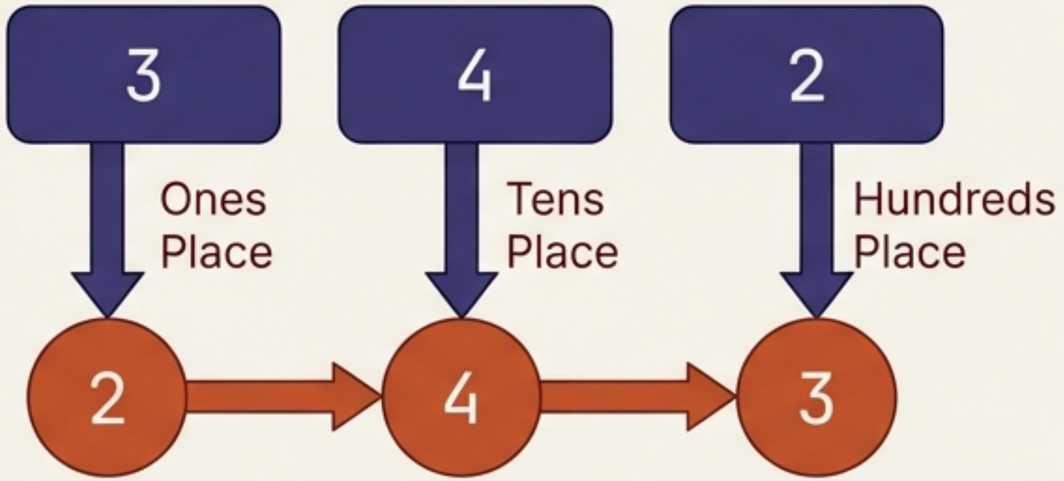
The Problem: Add two non-negative integers represented by linked lists.
The Twist: Digits are stored in **reverse order**.

NotebookLM

The Input Architecture

The Concept

```
Abstract Number: 342
Linked List: 2 -> 4 -> 3
```



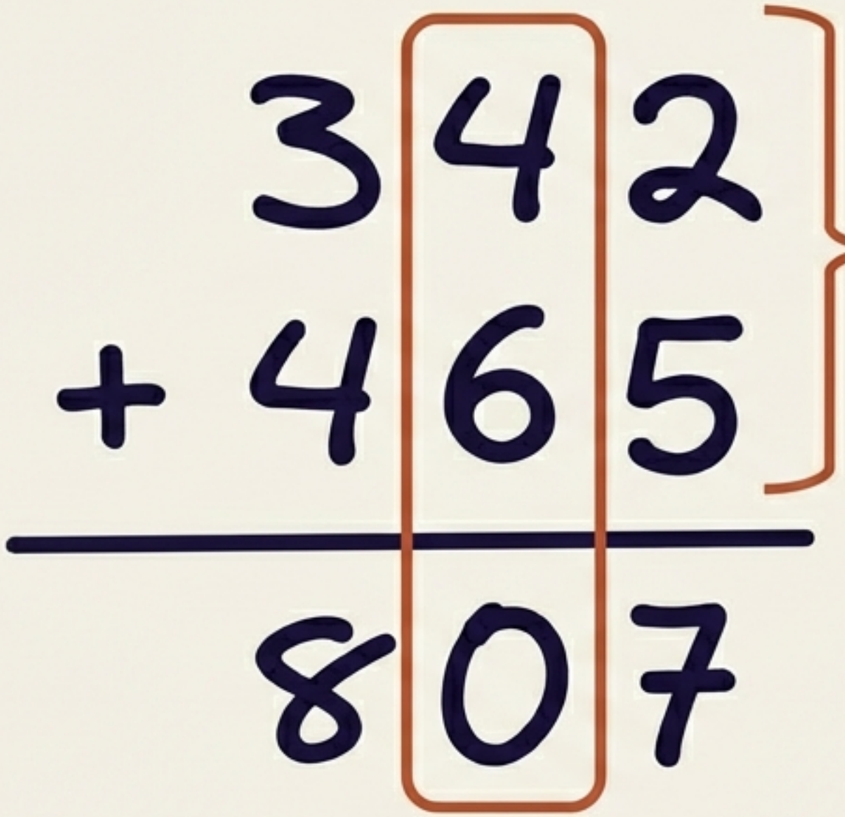
The Specs

Input Specifications

- **Input:** Two non-empty linked lists representing non-negative integers.
- **Constraint:** No leading zeros (e.g., the number 0 is just [0], not [0,0]).
- **The 'Reverse' Advantage:** Since the head of the list represents the 'ones' digit, the lists are implicitly aligned for standard addition from left to right. No traversing to the end required.
- **Goal:** Return the sum as a new linked list in the same reverse format.

NotebookLM

The Intuition: Elementary Math



Column Addition Logic:

1. **Sum:** The total of the column (4 + 6 = 10).
2. **Carry:** The overflow to the next column (1).

The Formula:

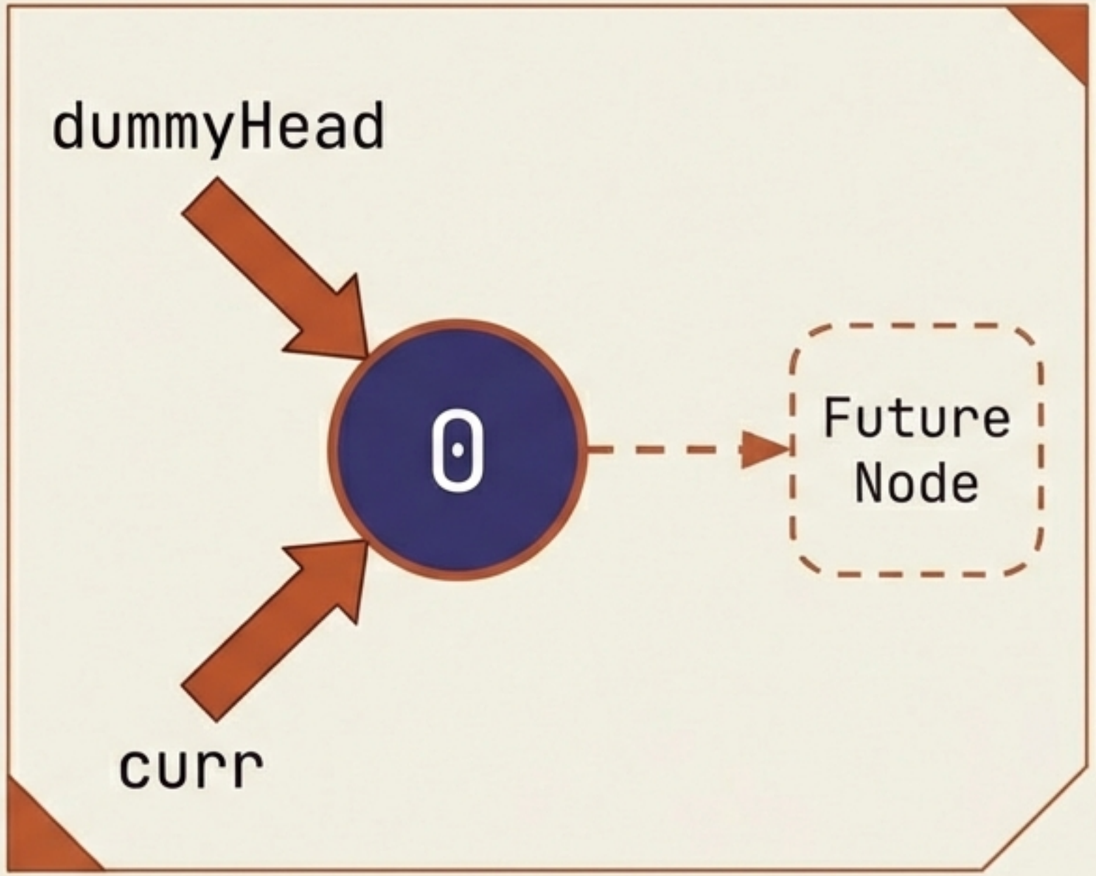
```
Total = Val1 + Val2 + Carry
Node Value = Total % 10 (Write this down)
New Carry = Total / 10 (Pass this over)
```

NotebookLM

The Setup: Why We Need a 'Dummy Head'

Building the result list presents a coding annoyance:
Creating the *very first* node requires different logic than creating the *subsequent* nodes (checking if “head is null”).

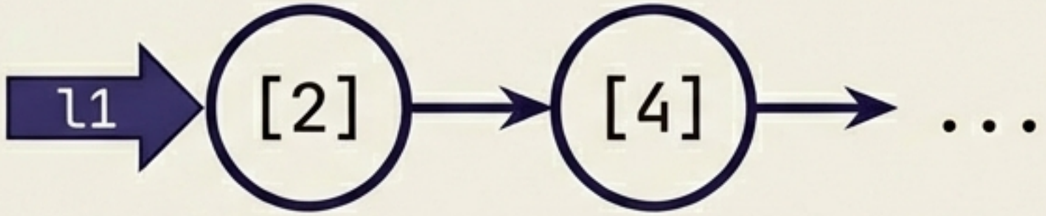
The Fix: Initialize a placeholder node called a “Dummy Head” with value 0. We build the result list attached to this node, then discard the dummy at the very end.



NotebookLM

Trace Step 1: Simple Addition

Input List 1

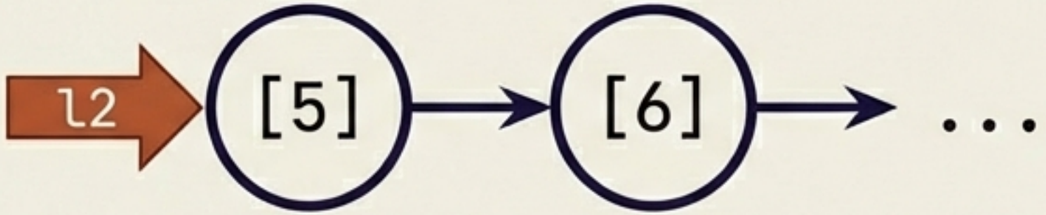


Variables

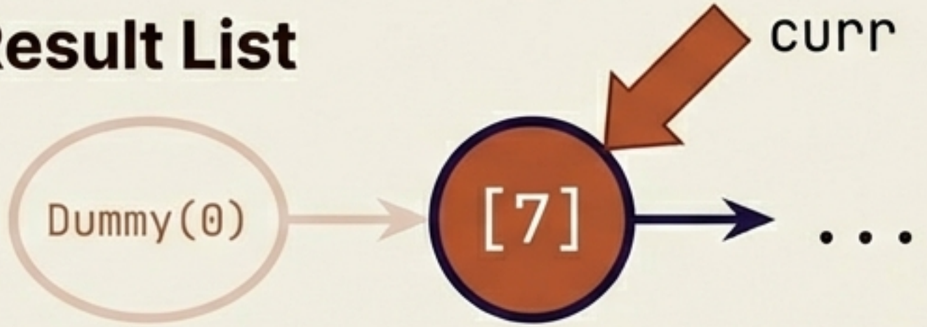
`Carry = 0`

Calculation: 2 + 5 + 0 = 7

Input List 2



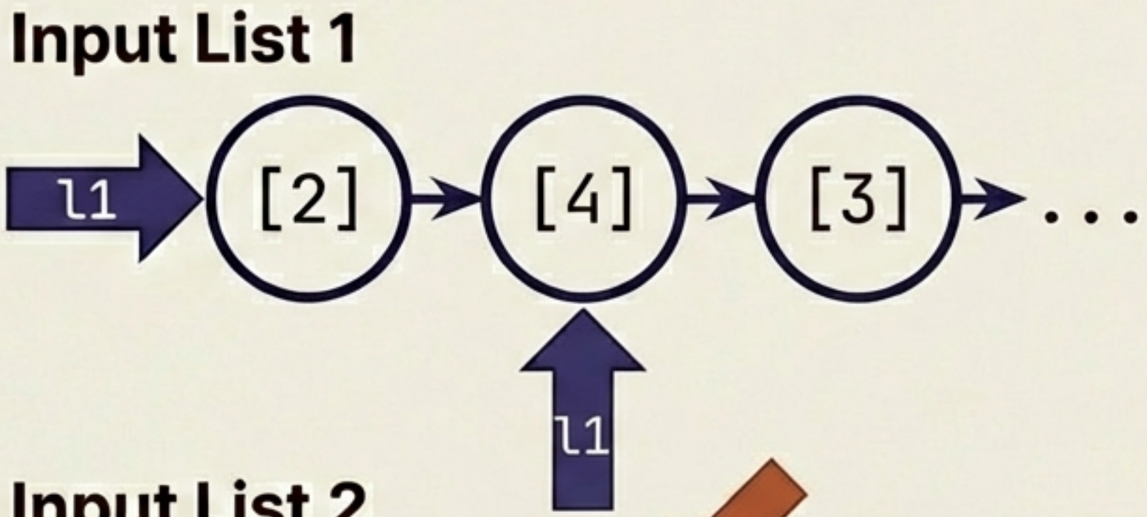
Result List



Resulting Node = 7. New Carry = 0.

NotebookLM

Trace Step 2: Generating the Carry



Variables

`Carry = 0`

Calculation: **4** (l1) + **6** (l2) + **0** (carry) = 10

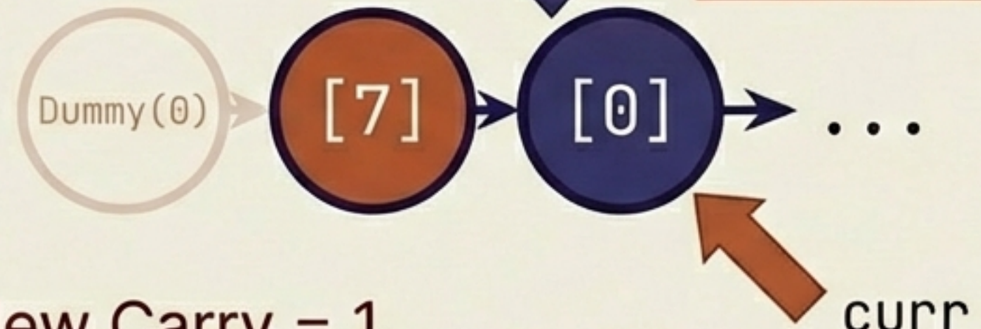
10

Carry Bucket

10 / 10 = **1** (New Carry)

Result List

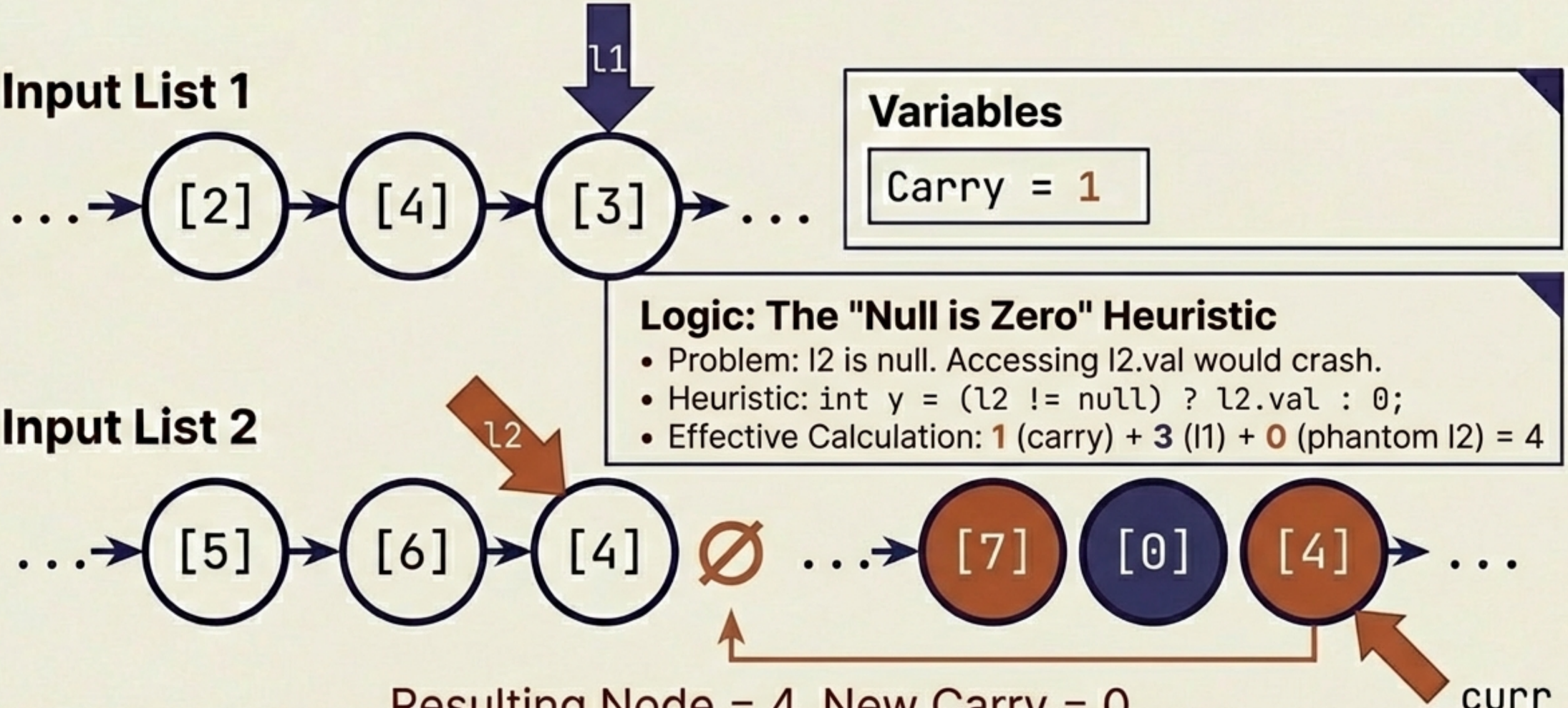
10 % 10 = 0



Resulting Node = 0. New Carry = 1.

NotebookLM

Trace Step 3: Uneven Lists (The "Null is Zero" Rule)

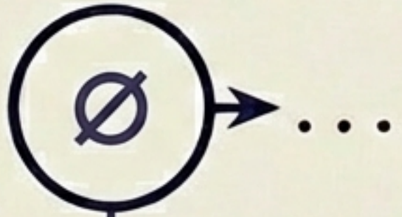


NotebookLM

Trace Step 4: The Final Carry

State Description

Input List 1: null
Input List 2: null
Carry Variable: 1



Loop Condition Check:

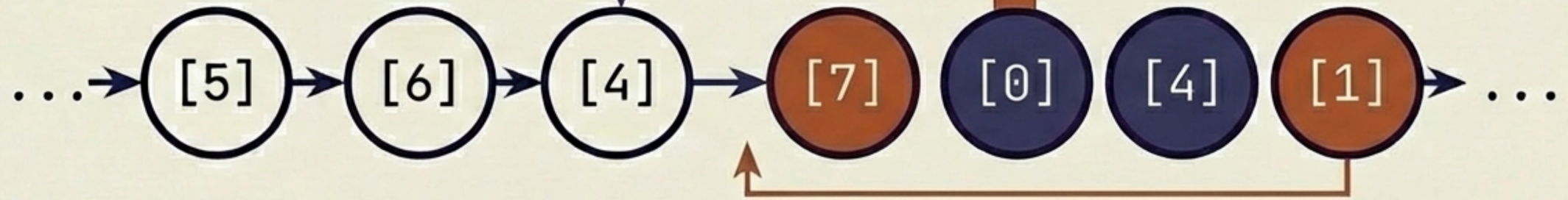
l1? No. l2? No. Carry? **YES.**

Variables

Carry = 1

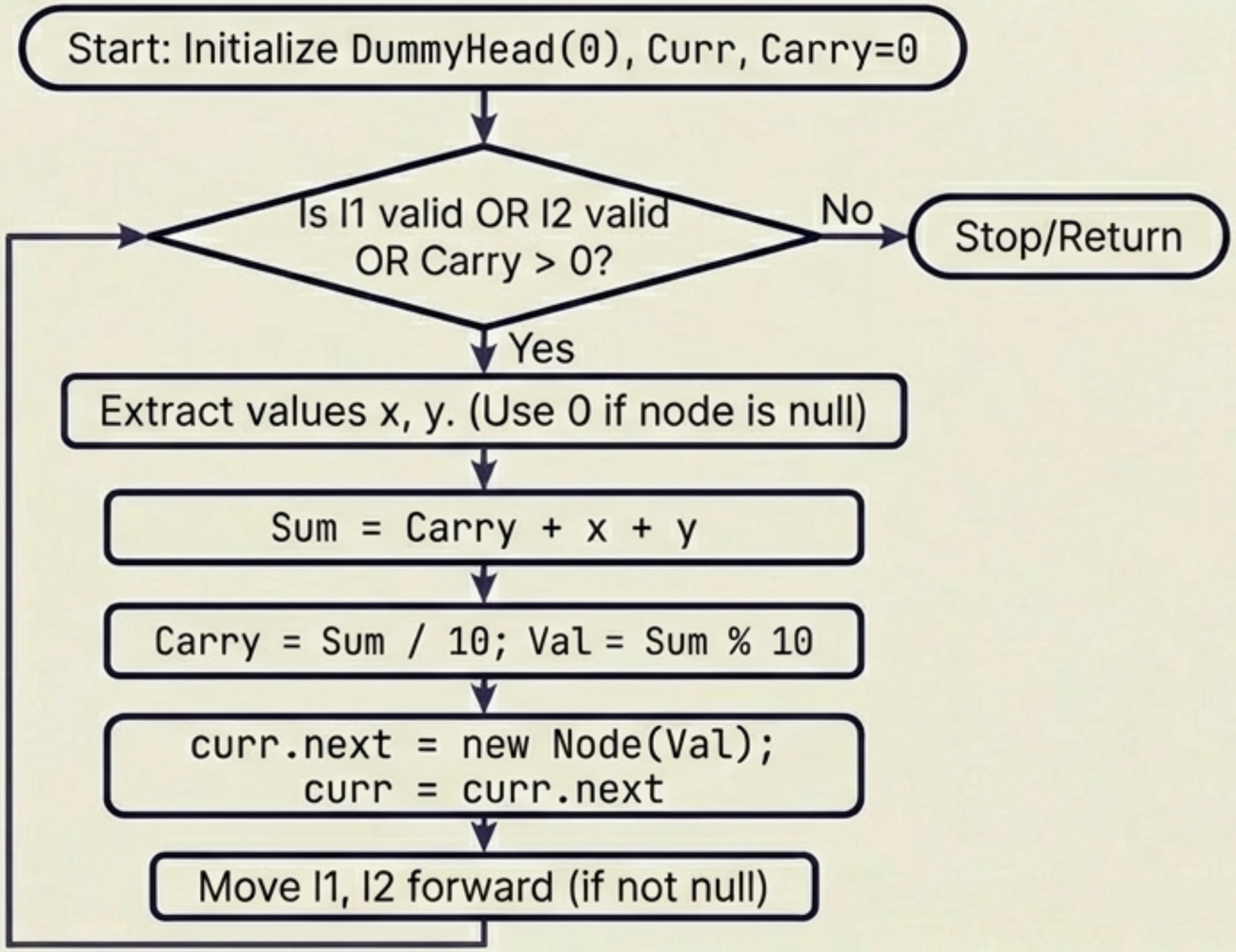
If we stopped when lists were empty, we would lose this digit (e.g., 50 + 50 = 100, not **00**). (Crimson Pro)

Result List



NotebookLM

The Algorithm Logic



NotebookLM

Implementation: The Setup & Loop

```
ListNode dummyHead = new ListNode(0);
ListNode curr = dummyHead;
int carry = 0;

// The Critical Condition
while (p1 != null || p2 != null || carry != 0) {
    // ... logic ...
}
```

Anchors the result list, avoiding initialization edge cases. (Inter & Crimson Pro)

This “OR” clause ensures the loop runs one last time if a carry remains after lists are exhausted. (Crimson Pro)

NotebookLM

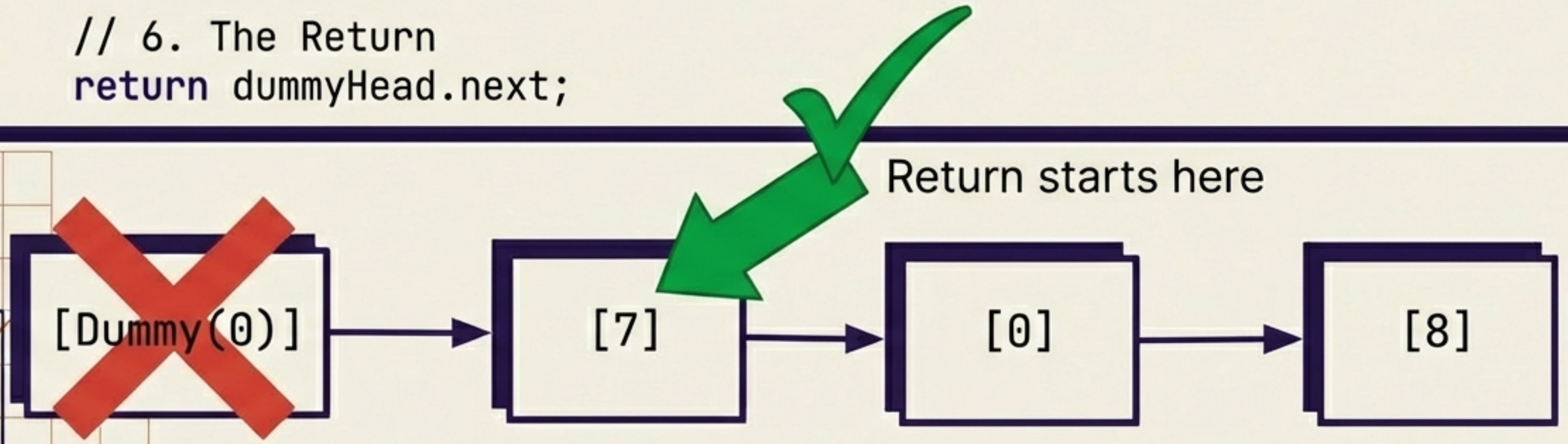
Implementation: The Core Math

```
// 1. Safe Value Extraction (Null = 0)
int x = (p1 != null) ? p1.val : 0;
int y = (p2 != null) ? p2.val : 0;

// 2. Calculate Sum
int sum = carry + x + y;

// 3. Update State
carry = sum / 10;                    // Integer division for next carry
curr.next = new ListNode(sum % 10); // Modulo for current digit

// 4. Move Result Pointer
curr = curr.next;
```

NotebookLM

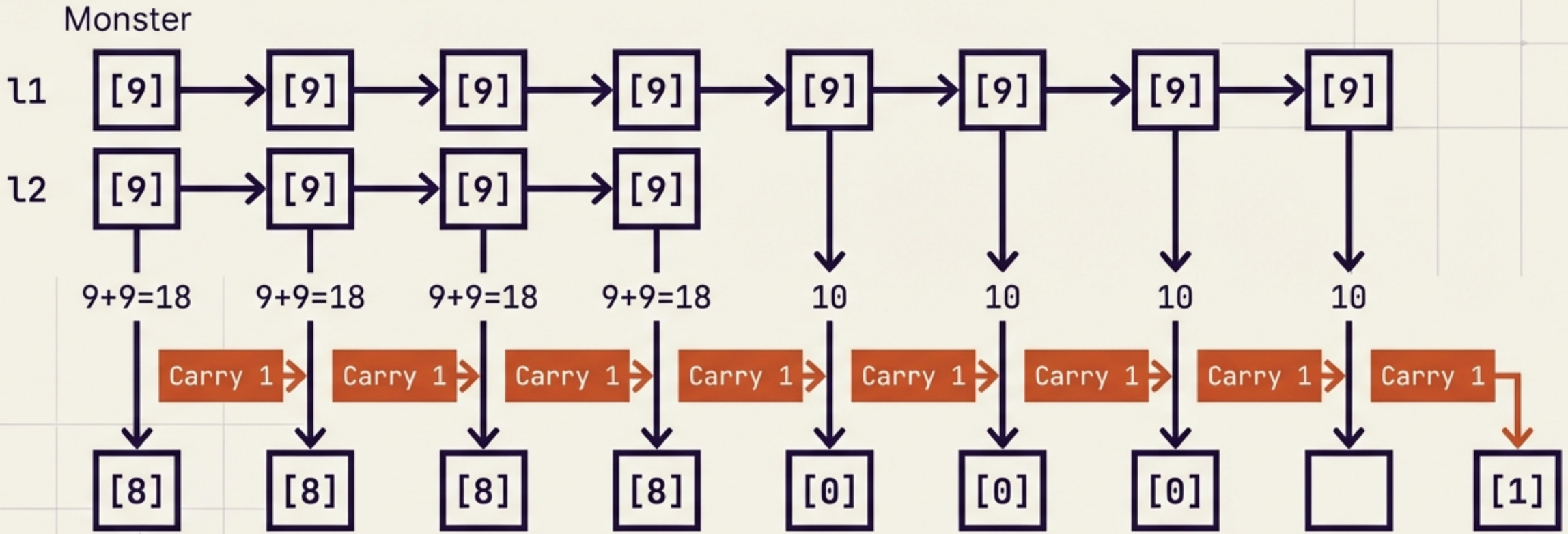
Implementation: Navigation & Return

```
// 5. Advance Input Pointers
if (p1 != null) p1 = p1.next;
   if (p2 != null) p2 = p2.next;
} // End While

// 6. The Return
return dummyHead.next;
```



NotebookLM

Edge Case Stress Test: Cascading Carries



[8, 9, 9, 9, 0, 0, 0, 1]

The logic naturally handles the “ripple” of +1 through the chain of 9s without recursion.

NotebookLM

Complexity Analysis

Time Complexity

O(max(M, N))

We iterate through the lists exactly once. The number of loop iterations is determined by the length of the longer list (plus potentially one extra operation for the final carry).

Space Complexity

O(max(M, N))

The length of the new result list is at most `max(M, N) + 1`. We do not use auxiliary data structures (like HashMaps), but the output list itself counts as space in this context.

NotebookLM

Summary & Key Takeaways

Math is Simple: It's just `Sum` and `Carry`, exactly like pen-and-paper addition.

Dummy Head: Using a placeholder node eliminates messy 'first node' initialization code.

Null as Zero: Treating `null` pointers as `0` allows a single clean loop for uneven lists.

The Final Check: Never forget the condition `|| carry != 0` to catch the final overflow.

A Medium difficulty problem solved with Elementary math.

NotebookLM